

Installation chez un nouveau client

Traceability 2.0 | fiche de procedure | mise a jour du 26/09/2026

PHASE 0 | La cle USB d'identifiants (une seule fois)

- ☐ Sur un Pi deja installe, creer le modele :

```
python3 ~/traceability-app/tools/finaliser.py --exemple
```
- ☐ Renseigner le jeton Telegram, le chat_id et, si besoin, les cles Tuya. Ces identifiants sont les MEMES sur toutes les machines.
- ☐ Copier le fichier **identifiants.txt** sur une cle USB. Elle servira a toutes les installations suivantes.

A PROTEGER Cette cle donne le controle du bot Telegram et l'accès au compte capteurs Tuya. La garder a l'atelier : ne jamais l'oublier chez un client.

PHASE 1 | Preparer la carte SD (~10 min)

- ☐ Carte de **16 Go minimum** (une installation complete occupe 7,5 Go). Acheter les cartes par lot identique.
- ☐ Raspberry Pi Imager : **Raspberry Pi OS avec bureau** (version proposee par default).
- ☐ **Modifier les reglages** avant d'ecrire :
 - ☐ • **Nom d'hote** = nom du client, en minuscules avec tirets (exemple : boucherie-durand)
 - ☐ • **WiFi** du magasin (reseau 2,4 GHz uniquement)
 - ☐ • **Nom d'utilisateur** : libre, le script s'adapte
- ☐ Ecrire l'image sur la carte.

ATTENTION Le nom d'hote est l'identifiant du Pi pour le verrouillage et les mises a jour a distance. Il doit etre unique et facile a reconnaitre.

PHASE 2 | Installer l'application (~45 min sans surveillance)

- ☐ Carte dans le Pi, ecran, alimentation. Demarrer et ouvrir un terminal.
- ☐ Lancer l'installation :

```
git clone https://github.com/halil6938/traceability-2.0.git
cd traceability-2.0
bash install.sh
sudo reboot
```
- ☐ Le script installe les paquets, les reglages camera (dtoverlay=ov5647,vcm) et le demarrage automatique. Le redemarrage est indispensable : sans lui, l'autofocus ne fonctionne pas.

GAIN DE TEMPS Plusieurs Pi peuvent etre installes **EN MEME TEMPS** : lancer la commande sur chacun, puis les laisser travailler en parallele. C'est le principal gain face au clonage de carte, qui monopolise le PC carte apres carte.

PHASE 3 | Finaliser la machine (~5 min)

- ☐ Brancher la cle USB d'identifiants, puis :

```
python3 ~/traceability-app/tools/finaliser.py
```

- ☐ Jeton Telegram et clés Tuya sont appliqués en une fois. Vérifier avec **--etat** ; un message Telegram confirme l'installation.
- ☐ **Orientation de l'écran** : Menu > Préférences > Screen Configuration, clic droit sur l'écran > Orientation > **Inverted** (180 degrés), puis Appliquer.
- ☐ Vérifier le nom de la machine :

```
hostname
```
- ☐ S'il est incorrect :

```
bash ~/traceability-app/tools/set_client.sh boucherie-durand
sudo reboot
```

PHASE 4 | Configuration dans l'application (~15 min)

a) Les appareils (frigos et congélateurs)

- ☐ L'assistant du premier démarrage demande d'en créer un : nom, température MIN, température MAX. Ces seuils déclenchent les alertes.
- ☐ Pour les suivants : **Paramètres > + Ajouter**.

b) Les capteurs de température (Paramètres > Capteurs temp.)

- ☐ **Ajouter BLE** : capteurs allumés et proches. Ils s'affichent avec leur température actuelle, ce qui permet d'identifier lequel est dans quel frigo. Le bouton **Adresse** permet une saisie manuelle.
- ☐ **Ajouter WiFi** (capteurs Tuya) : le capteur doit d'abord être appairé dans l'application Smart Life du fournisseur, sinon il n'apparaît pas dans la liste.
- ☐ **Assigner** chaque capteur à son appareil, puis **Tester** pour vérifier que les températures remontent.

c) La réception (Réception > Fournisseurs)

- ☐ Saisir les fournisseurs du client, avec si besoin la **température maximale acceptée** (une livraison au-dessus s'affiche en rouge).
- ☐ **Pistolet BLE** puis **Detecter** : appuyer sur la gachette pendant la recherche. La MAC peut aussi être saisie à la main.

d) La caméra (Paramètres > Test caméra)

- ☐ Placer un ticket imprimé bien éclairé à sa distance définitive, puis **Calibrer** (environ 10 s, sans rien bouger).
- ☐ Vérifier ensuite un scan réel, et la lisibilité de la photo.

ATTENTION La mise au point dépend du montage : la calibration est à refaire sur CHAQUE installation.

PHASE 5 | Enregistrer le client pour le contrôle à distance

- ☐ Sur GitHub, dossier **devices/**, créer le fichier **<nom-du-client>.json** en copiant **_modele.json** :

```
{ "locked": false, "message": "", "update": "auto",
  "config": { "NOM_MAGASIN": "Boucherie Durand" } }
```
- ☐ Pour bloquer ce client : passer **locked** à true et écrire un message. Effet en 20 secondes environ ; le blocage résiste au redémarrage et à une coupure internet.
- ☐ **update** : auto (mises à jour la nuit, recommande), now (immédiat) ou off (fige la version).
- ☐ **NOM_MAGASIN** : nom imprimé en tête des exports PDF (sinon, le nom de la machine).

MEMO | Depannage et verifications

Version installee et nom du Pi : en bas de l'ecran **Parametres**.

```
python3 ~/traceability-app/tools/finaliser.py --etat # identifiants
sudo systemctl restart traceability # redemarrer l'application
journalctl -u traceability -n 40 # journal de l'application
cat ~/traceability/logs/update.log # mises a jour
cat ~/traceability/logs/ble_thermo.log # pistolet infrarouge
python3 ~/traceability-app/tools/focus_probe.py # test du focus
```

Le code se met a jour tout seul depuis GitHub : rien a transferer sur les Pi apres une correction.

Signe de vie Telegram : chaque Pi annonce son installation, ses mises a jour, et envoie un message quotidien. Un Pi silencieux est hors ligne.

L'ecran se met en veille apres 10 minutes sans contact. Le premier contact rallume seulement : il ne declenche aucun bouton. Delai reglable a distance (SCREEN_OFF_S).

Quitter l'application : appui long (3 s) sur la ligne de version en bas de **Parametres**. Elle ne se relance qu'au prochain redemarrage du Pi.

« Heure non verifiee » en orange dans le menu : le Pi n'a pas pu se mettre a l'heure (pas d'internet au demarrage). Verifier le WiFi ; un module horloge RTC evite ce cas.